Voting Verification Codebase

File: README.md

```
# Voting Verification System

A secure, client-side, single-page web application for verifying and storing voting records locally.
Designed with a "Mobile-First" approach, featuring a modern Glassmorphism dark UI.

## Features

- **Operator Authentication**: Simple login to prevent unauthorized access.
- **Voter Storage**: Persists voter data in the browser's `localStorage`.
- **Duplicate Prevention**: Prevents double-voting by checking the **First Name + Voter ID** combination.
- **Data Export**: Backup your voting database to a `.json` file.
- **Data Import**: Restore or merge voting data from a `.json` file.
- **Real-time Stats**: Always shows the total count of unique stored positions.

## How it Works

### 1. Data Storage
All data is stored in the browser's **Local Storage** under the key `votingRecords`.
Each record is a JSON object containing:
- `firstName`: (string, lowercased for uniqueness)
- `lastName`: (string, optional)
- `voterId`: (string, treated as case-insensitive for uniqueness)
- `timestamp`: (ISO string of when the vote was cast)

### 2. Voting Process
1. Enter the voter's details.
2. Click **Submit Vote**.
3. If the **First Name + Voter ID** combination is new, the vote is saved.
4. If the exact combination exists, an error is shown ("Already voted!").

### 3. Export Data
Clicking the **Export Data** button generates a file named `voting_data_export.json` containing the full array of voting records. This happens entirely in the browser using the `Blob` API.

### 4. Import Data
Clicking **Import Data** allows you to select a `.json` file.
**Merge Logic:**
- The app reads the uploaded file.
- It iterates through each record.
- **Rules:**
      - If a voter with the same `firstName` AND `voterId` exists in the local database, the imported record is **skipped** (existing data takes precedence).
    - If the combination is new, it is **added**.
- This prevents accidental overwrites or duplicates during data consolidation.

### 5. Resetting / Clearing Data
Since there is no "Delete" button for security:
- To **CLEAR ALL DATA**: Open your browser's Developer Tools (F12) -> Application -> Local Storage, and delete the `votingRecords` key.
- Alternatively, clear your browser's "Site Data" for the page.

## Technical Details
- **Stack**: HTML5, CSS3, Vanilla JavaScript (ES6+).
- **No Backend**: Runs 100% on the client device.
- **No Libraries**: Zero dependencies.

## Setup
```

Voting Verification Codebase

Simply open `index.html` in any modern web browser. No server required.

File: app.py

```
from flask import Flask, request, jsonify, send_from_directory
import sqlite3
import os
from datetime import datetime

app = Flask(__name__, static_folder='.')

DB_FILE = 'votes.db'

def init_db():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS voters (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT,
            voter_id TEXT,
            timestamp TEXT
        )
    ''')
    conn.commit()
    conn.close()

init_db()

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/<path:path>')
def static_files(path):
    return send_from_directory('.', path)

@app.route('/api/login', methods=['POST'])
def login():
    data = request.json
    if data.get('username') == 'admin' and data.get('password') == 'admin123':
        return jsonify({"success": True})
    return jsonify({"success": False, "message": "Invalid Credentials"}), 401

@app.route('/api/stats', methods=['GET'])
def stats():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('SELECT COUNT(*) FROM voters')
    count = cursor.fetchone()[0]
    conn.close()
    return jsonify({"count": count})

@app.route('/api/vote', methods=['POST'])
def vote():
    data = request.json
    first_name = data.get('firstName', '').strip().lower()
    last_name = data.get('lastName', '').strip()
    voter_id = data.get('voterId', '').strip().lower()
```

```
    if not first_name:
        return jsonify({"success": False, "message": "First name is required"}), 400

    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    # Check duplicate
     cursor.execute('SELECT * FROM voters WHERE LOWER(first_name) = ? AND LOWER(voter_id) = ?',
(first_name, voter_id))
    if cursor.fetchone():
        conn.close()
         return jsonify({"success": False, "message": f'Error: "{first_name}" with ID "{voter_id}"
has already voted!'}), 400

    timestamp = datetime.utcnow().isoformat() + "Z"
    cursor.execute('''
        INSERT INTO voters (first_name, last_name, voter_id, timestamp)
        VALUES (?, ?, ?, ?)
    ''', (first_name, last_name, voter_id, timestamp))
    conn.commit()
    conn.close()

    return jsonify({"success": True, "message": "Vote Recorded Successfully"})

@app.route('/api/export', methods=['GET'])
def export_data():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('SELECT first_name, last_name, voter_id, timestamp FROM voters')
    rows = cursor.fetchall()
    conn.close()

    records = []
    for row in rows:
        records.append({
            "firstName": row[0],
            "lastName": row[1],
            "voterId": row[2],
            "timestamp": row[3]
        })
    return jsonify(records)

@app.route('/api/import', methods=['POST'])
def import_data():
    data = request.json
    if not isinstance(data, list):
        return jsonify({"success": False, "message": "Invalid format"}), 400

    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    added_count = 0
    skipped_count = 0

    for record in data:
        first_name = record.get('firstName', '').strip().lower()
        if not first_name:
            continue

        last_name = record.get('lastName', '').strip()
        voter_id = record.get('voterId', '').strip().lower()
        timestamp = record.get('timestamp', datetime.utcnow().isoformat() + "Z")
```

```
        cursor.execute('SELECT * FROM voters WHERE LOWER(first_name) = ? AND LOWER(voter_id) = ?',
(first_name, voter_id))
        if cursor.fetchone():
            skipped_count += 1
        else:
            cursor.execute('''
                INSERT INTO voters (first_name, last_name, voter_id, timestamp)
                VALUES (?, ?, ?, ?)
            ''', (first_name, last_name, voter_id, timestamp))
            added_count += 1

    conn.commit()
    conn.close()

    return jsonify({
        "success": True,
        "message": f"Imported {added_count} records. Skipped {skipped_count} duplicates."
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Voting Verification System</title>
    <meta name="description" content="Secure local-storage based voting verification system">
                                                                                                <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>Vote Verification System</h1>
            <p class="subtitle">Secure Local-Storage Validation</p>
        </header>

        <div id="loginSection" class="card">
                        <h2  style="font-size:1.1rem;  margin-bottom:1rem;  color:var(--text-main);
text-align:center;">Operator Login</h2>
            <form id="loginForm">
                <div class="form-group">
                    <label for="username">Username</label>
                     <input type="text" id="username" name="username" placeholder="Enter username"
autocomplete="off" required>
                </div>
                <div class="form-group">
                    <label for="password">Password</label>
                          <input type="password" id="password" name="password" placeholder="Enter
password" required>
                </div>
                <button type="submit" class="btn-primary">Login</button>
            </form>
            <div id="loginMessage" class="message-box"></div>
```

```
        </div>

        <div id="appContent" class="hidden">
            <div class="card" style="margin-bottom: 1.5rem;">
                <div class="stats-container">
                    <span class="stat-label">Total Unique Voters</span>
                    <span class="stat-value" id="statsCount">0</span>
                </div>
            </div>

            <div class="card" style="margin-bottom: 1.5rem;">
                    <h2 style="font-size:1.1rem; margin-bottom:1rem; color:var(--text-main);">Cast
Vote</h2>
                <form id="votingForm">
                    <div class="form-group">
                                                          <label  for="firstName">First  Name  <span
style="color:var(--error-color)">*</span></label>
                                    <input  type="text"  id="firstName"  name="firstName"  required
placeholder="e.g. John" autocomplete="off">
                    </div>

                    <div class="form-group">
                        <label for="lastName">Last Name</label>
                           <input type="text" id="lastName" name="lastName" placeholder="e.g. Doe"
autocomplete="off">
                    </div>

                    <div class="form-group">
                        <label for="voterId">Voter ID / Email</label>
                             <input type="text" id="voterId" name="voterId" placeholder="ID-123 or
email@example.com" autocomplete="off">
                    </div>

                    <button type="submit" class="btn-primary">Submit Vote</button>
                </form>

                <div id="statusMessage" class="message-box"></div>
            </div>

            <div class="card">
                    <h2 style="font-size:1.1rem; margin-bottom:1rem; color:var(--text-main);">Data
Management</h2>
                <div class="actions-grid">
                    <button id="exportBtn" class="btn-outline">Export Data</button>
                    <button id="importBtn" class="btn-outline">Import Data</button>
                           <button id="logoutBtn" class="btn-outline" style="grid-column: span 2;
border-color: var(--error-color); color: var(--error-color);">Logout</button>
                </div>
                <input type="file" id="fileInput" accept=".json" class="hidden">
                <div id="importStatus" class="message-box"></div>
            </div>

            <div class="footer">
                Backend Database Persistence Enabled
            </div>
        </div>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

Voting Verification Codebase

File: script.js

```
const form = document.getElementById('votingForm');
const firstNameInput = document.getElementById('firstName');
const lastNameInput = document.getElementById('lastName');
const voterIdInput = document.getElementById('voterId');
const statusMessage = document.getElementById('statusMessage');
const statsCount = document.getElementById('statsCount');
const exportBtn = document.getElementById('exportBtn');
const importBtn = document.getElementById('importBtn');
const fileInput = document.getElementById('fileInput');
const importStatus = document.getElementById('importStatus');
const logoutBtn = document.getElementById('logoutBtn');
const loginSection = document.getElementById('loginSection');
const appContent = document.getElementById('appContent');
const loginForm = document.getElementById('loginForm');
const loginMessage = document.getElementById('loginMessage');
const usernameInput = document.getElementById('username');
const passwordInput = document.getElementById('password');

const API_BASE = '/api';

async function updateStats() {
    try {
        const res = await fetch(`${API_BASE}/stats`);
        const data = await res.json();
        statsCount.textContent = data.count;
    } catch (err) {
        console.error("Error fetching stats", err);
    }
}

function showMessage(element, msg, isError = false) {
    element.textContent = msg;
    element.className = 'message-box';
    element.classList.add(isError ? 'message-error' : 'message-success');
    element.style.display = 'block';

    setTimeout(() => {
        element.style.display = 'none';
    }, 3000);
}

form.addEventListener('submit', async (e) => {
    e.preventDefault();

    const firstName = firstNameInput.value.trim();
    const lastName = lastNameInput.value.trim();
    const voterId = voterIdInput.value.trim();

    if (!firstName) {
        showMessage(statusMessage, "First name is required.", true);
        return;
    }

    try {
        const res = await fetch(`${API_BASE}/vote`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ firstName, lastName, voterId })
        });
```

```
        const data = await res.json();

        if (data.success) {
            showMessage(statusMessage, data.message, false);
            form.reset();
            updateStats();
        } else {
            showMessage(statusMessage, data.message, true);
        }
    } catch (err) {
        showMessage(statusMessage, "Server error. Please try again.", true);
    }
});

exportBtn.addEventListener('click', async () => {
    try {
        const res = await fetch(`${API_BASE}/export`);
        const data = await res.json();

        if (data.length === 0) {
            showMessage(importStatus, "No data to export.", true);
            return;
        }

        const dataStr = JSON.stringify(data, null, 2);
        const blob = new Blob([dataStr], { type: "application/json" });
        const url = URL.createObjectURL(blob);

        const a = document.createElement('a');
        a.href = url;
        a.download = "voting_data_export.json";
        document.body.appendChild(a);
        a.click();
        document.body.removeChild(a);
        URL.revokeObjectURL(url);
    } catch (err) {
        showMessage(importStatus, "Failed to export data.", true);
    }
});

importBtn.addEventListener('click', () => {
    fileInput.click();
});

fileInput.addEventListener('change', (e) => {
    const file = e.target.files[0];
    if (!file) return;

    const reader = new FileReader();

    reader.onload = async function (event) {
        try {
            const importedData = JSON.parse(event.target.result);

            if (!Array.isArray(importedData)) {
                throw new Error("Invalid format");
            }

            const res = await fetch(`${API_BASE}/import`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
```

```
                body: JSON.stringify(importedData)
            });

            const data = await res.json();

            if (data.success) {
                showMessage(importStatus, data.message, false);
                updateStats();
            } else {
                showMessage(importStatus, data.message, true);
            }

        } catch (err) {
            console.error(err);
            showMessage(importStatus, "Failed to import: Invalid JSON file.", true);
        }

        fileInput.value = '';
    };

    reader.readAsText(file);
});

loginForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const username = usernameInput.value;
    const password = passwordInput.value;

    try {
        const res = await fetch(`${API_BASE}/login`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username, password })
        });

        const data = await res.json();

        if (data.success) {
            loginSection.classList.add('hidden');
            appContent.classList.remove('hidden');
            passwordInput.value = '';
            updateStats();
        } else {
            showMessage(loginMessage, data.message || "Invalid Credentials", true);
        }
    } catch (err) {
        showMessage(loginMessage, "Server error during login.", true);
    }
});

logoutBtn.addEventListener('click', () => {
    appContent.classList.add('hidden');
    loginSection.classList.remove('hidden');
    usernameInput.value = '';
    passwordInput.value = '';
});
```

File: style.css

```
:root {
    --bg-color: #0f172a;
```

```
    --card-bg: rgba(30, 41, 59, 0.7);
    --card-border: rgba(148, 163, 184, 0.1);
    --primary-color: #3b82f6;
    --primary-hover: #2563eb;
    --success-color: #10b981;
    --error-color: #ef4444;
    --text-main: #f8fafc;
    --text-secondary: #94a3b8;
    --input-bg: rgba(15, 23, 42, 0.6);
    --glass-shadow: 0 4px 30px rgba(0, 0, 0, 0.1);
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: 'Inter', sans-serif;
    background-color: var(--bg-color);
    background-image:
        radial-gradient(at 0% 0%, rgba(59, 130, 246, 0.15) 0px, transparent 50%),
        radial-gradient(at 100% 0%, rgba(139, 92, 246, 0.15) 0px, transparent 50%),
        radial-gradient(at 100% 100%, rgba(16, 185, 129, 0.15) 0px, transparent 50%),
        radial-gradient(at 0% 100%, rgba(239, 68, 68, 0.15) 0px, transparent 50%);
    background-attachment: fixed;
    color: var(--text-main);
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 2rem 1rem;
}

.container {
    width: 100%;
    max-width: 480px;
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
}

header {
    text-align: center;
    margin-bottom: 0.5rem;
}

h1 {
    font-size: 1.75rem;
    font-weight: 700;
    background: linear-gradient(to right, #60a5fa, #a78bfa);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    letter-spacing: -0.025em;
}

.subtitle {
    color: var(--text-secondary);
    font-size: 0.875rem;
    margin-top: 0.25rem;
}
```

```
.card {
    background: var(--card-bg);
    backdrop-filter: blur(12px);
    -webkit-backdrop-filter: blur(12px);
    border: 1px solid var(--card-border);
    border-radius: 16px;
    padding: 1.5rem;
    box-shadow: var(--glass-shadow);
}

.stats-container {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.stat-label {
    font-size: 1rem;
    color: var(--text-secondary);
    font-weight: 500;
}

.stat-value {
    font-size: 1.5rem;
    font-weight: 700;
    color: var(--text-main);
}

.form-group {
    margin-bottom: 1rem;
}

label {
    display: block;
    margin-bottom: 0.5rem;
    font-size: 0.875rem;
    font-weight: 500;
    color: var(--text-secondary);
}

input {
    width: 100%;
    padding: 0.75rem 1rem;
    border-radius: 8px;
    border: 1px solid var(--card-border);
    background: var(--input-bg);
    color: white;
    font-size: 1rem;
    font-family: inherit;
    transition: all 0.2s ease;
}

input:focus {
    outline: none;
    border-color: var(--primary-color);
    box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.2);
}

button {
    width: 100%;
    padding: 0.75rem;
```

```
    border-radius: 8px;
    border: none;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.2s ease;
    position: relative;
    overflow: hidden;
}

button:active {
    transform: scale(0.98);
}

.btn-primary {
    background: linear-gradient(135deg, var(--primary-color), var(--primary-hover));
    color: white;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
}

.btn-primary:hover {
    opacity: 0.9;
}

.btn-outline {
    background: transparent;
    border: 1px solid var(--text-secondary);
    color: var(--text-secondary);
    font-size: 0.875rem;
    padding: 0.5rem;
}

.btn-outline:hover {
    border-color: var(--text-main);
    color: var(--text-main);
    background: rgba(255, 255, 255, 0.05);
}

.actions-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1rem;
    margin-top: 0.5rem;
}

.message-box {
    margin-top: 1rem;
    padding: 0.75rem;
    border-radius: 8px;
    font-size: 0.875rem;
    display: none;
    text-align: center;
    animation: fadeIn 0.3s ease;
}

.message-success {
    background: rgba(16, 185, 129, 0.2);
    color: #6ee7b7;
    border: 1px solid rgba(16, 185, 129, 0.3);
}

.message-error {
```

```
    background: rgba(239, 68, 68, 0.2);
    color: #fca5a5;
    border: 1px solid rgba(239, 68, 68, 0.3);
}

.footer {
    text-align: center;
    color: rgba(148, 163, 184, 0.5);
    font-size: 0.75rem;
    margin-top: 2rem;
}

@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(-10px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.hidden {
    display: none;
}
```

File: requirements.txt

```
flask==3.0.0
```

File: .gitignore